

Document AI

What is Document AI

Document AI is a Snowflake AI feature that uses Arctic-TILT, a proprietary large language model (LLM), to extract data from documents. Document AI processes documents of various formats and extracts information from both text-heavy paragraphs and the content in a graphical form, such as logos, handwritten text (signatures), tables, or checkmarks. With Document AI, you can prepare pipelines for continuous processing of new documents of a specific type, such as invoices or finance statements. You can extract information from entities (in a form of a single value or a list of values), or tables based on the list of specified columns.

Document AI provides both zero-shot extraction and fine-tuning. Zero-shot means that the foundation model can locate and extract information specific to a document type, even if the model has never seen the document before. This is because the foundation model is trained on a large volume of various documents, so the model broadly understands the type of document being processed.

Additionally, you can fine-tune the Snowflake Arctic-TILT model to improve your results by training the model on the documents specific to your use case. The fine-tuned model (including the training data used) is available only to you and is not shared with other Snowflake customers.

When to use Document AI

Document AI is best used when:

- You want to turn unstructured data from documents into structured data in tables.
- You want to create pipelines for continuous processing of new documents of a specific type.
- Business users with domain knowledge prepare the model, and the data engineers working with SQL prepare pipelines to automate the processing of new documents.

How Document AI works

Working with Document AI is divided into two phases:

- Preparing a Document AI model build

You can think of the model build as representing a single type of a document or a use case; for example, a model build for extracting information from invoice documents. The Document AI model build includes the model, the data values to be extracted, and the documents uploaded to test and train the model.

You prepare the model build through a Document AI user interface in Snowsight. The interface lets you create a model build, upload documents to test and train the model, define data values (information to be extracted) by asking questions using natural language, evaluate the model, and publish the model build or fine-tune the model to improve the results.

Create a document processing pipeline with document AI

Introduction

With Document AI, you can process documents of various formats, and extract information from both text-heavy paragraphs and the images that contain text, such as logos, handwritten text (signatures), or checkmarks. This tutorial introduces you to Document AI by setting up the required objects and privileges, and creating a Document AI model build to use in a processing pipeline.

The tutorial uses the Snowsight web interface. For the steps that require using SQL, you can use any Snowflake client that supports executing SQL.

What you will learn

In this tutorial, you will learn how to:

- Set up the objects and privileges required to work with Document AI.
- Prepare a Document AI model build using Document AI user interface in Snowsight to extract data from unstructured documents.
- Create a pipeline for continuous processing of new documents in stage, using a Document AI model build, and streams and tasks.

Prerequisites

The following prerequisites are required to complete this tutorial:

- You must connect as a user that has the ACCOUNTADMIN role which is used to create a custom role used in this tutorial and grant the new role required privileges.

- You must have a Snowflake account in one of the commercial regions supported for Document AI. For more information about supported regions, see [Document AI availability](#).
- You must have a warehouse ready to use with Document AI.

Set up the required objects and privileges

In this section, you will:

- Create a database and schema to contain the Document AI model build.
- Create a custom role to prepare a Document AI model build and a document processing pipeline.
- Grant the required privileges to the custom role.

Create a database, schema, and custom role

To create a database, schema, and a role to work with Document AI, do the following:

1. Create a database and schema in which to create a Document AI model build:

```
CREATE DATABASE doc_ai_db;
```

```
CREATE SCHEMA doc_ai_db.doc_ai_schema;
```

2. Create custom role doc_ai_role to prepare the Document AI model build and to create processing pipelines:

```
USE ROLE ACCOUNTADMIN;
```

```
CREATE ROLE doc_ai_role;
```

Grant the required privileges

To grant the privileges required to work with Document AI, do the following:

1. Grant the SNOWFLAKE.DOCUMENT_INTELLIGENCE_CREATOR database role to the doc_ai_role role:

GRANT DATABASE ROLE SNOWFLAKE.DOCUMENT_INTELLIGENCE_CREATOR TO ROLE doc_ai_role;

2. Grant warehouse usage and operating privileges to the doc_ai_role role:

GRANT USAGE, OPERATE ON WAREHOUSE <your_warehouse> TO ROLE doc_ai_role;

3. Grant the privileges to use the database and schema you created to the doc_ai_role:

GRANT USAGE ON DATABASE doc_ai_db TO ROLE doc_ai_role;

GRANT USAGE ON SCHEMA doc_ai_db.doc_ai_schema TO ROLE doc_ai_role;

4. Grant the create stage privilege on the schema to the doc_ai_role role to store the documents for extraction:

GRANT CREATE STAGE ON SCHEMA doc_ai_db.doc_ai_schema TO ROLE doc_ai_role;

5. Grant the privileges to create model builds (instances of the DOCUMENT_INTELLIGENCE class) to the doc_ai_role role:

GRANT CREATE SNOWFLAKE.ML.DOCUMENT_INTELLIGENCE ON SCHEMA doc_ai_db.doc_ai_schema TO ROLE doc_ai_role;

GRANT CREATE MODEL ON SCHEMA doc_ai_db.doc_ai_schema TO ROLE doc_ai_role;

6. Grant the privileges required to create a processing pipeline using streams and tasks to the doc_ai_role role:

GRANT CREATE STREAM, CREATE TABLE, CREATE TASK, CREATE VIEW ON SCHEMA doc_ai_db.doc_ai_schema TO ROLE doc_ai_role;

GRANT EXECUTE TASK ON ACCOUNT TO ROLE doc_ai_role;

7. Grant the doc_ai_role to tutorial user for use in the next steps of the tutorial:

GRANT ROLE doc_ai_role TO USER <your_user_name>;

Prepare a Document AI model build

In this section, you will prepare a Document AI model build by creating the model build and uploading documents to test the model.

The Document AI model build represents a single type of the document. For this tutorial, you will create a model build for extracting information from inspection reviews. The Document AI model build includes the model, the data values to be extracted, and the documents uploaded to test the model.

Create a Document AI model build

To create a Document AI model build, do the following:

1. Sign in to [Snowsight](#).
2. In the navigation menu, select **AI & ML » Document AI**.
3. Select a warehouse.
4. Select **+ Build**.
5. In the dialog that appears, enter `inspection_reviews` as a name for your model build, and select the location (`doc_ai_db` database and `doc_ai_schema` schema).
6. Select **Create**.

Upload documents to the Document AI model build

To upload documents to the newly created Document AI model build, do the following:

1. To obtain the documents required for the tutorial to test the model build, download the [zip file](#) to your local file system.
2. Unzip the content, which includes PDF documents.
3. In the `inspection_reviews` model build, select the **Build Details** tab.
4. Select **Upload documents**.
5. Select **Browse** or drag the documents you downloaded.
6. Select **Upload**.

Define data values and review the results

In this section, you will define data values by asking the Document AI model questions in natural language. You will then review the answers that the model provides.

Data values are the information you want to extract from documents. A value consists of a value name and a question asked in natural language.

To define values for the Document AI model build:

1. In the inspection_reviews model build, select the **Build Details** tab.
2. Select **Define values**.
3. In the **Documents review** view, select **+ Value**.
4. For each document, enter the following pairs of value names and questions:
 - inspection_date: What is the inspection date?
 - inspection_grade: What is the grade?
 - inspector: Who performed the inspection?
 - list_of_units: What are all the units?
5. For each document and data value, review the answers that the model provides:
 - If the answer is correct, select the checkmark.
 - If the answer is incorrect, enter the correct value manually.

Publish a Document AI model build

In this **section**, you will publish the Document AI model build to use it for extraction in processing pipelines. Publishing the model build enables using the latest version of the model build in production.

To publish the model build, do the following:

1. In the inspection_reviews model build, select the **Build Details** tab.
2. Under **Model accuracy**, select **Publish version**.
3. In the dialog that appears, select **Publish** to confirm.

Create a document processing pipeline

In this section, you will create a processing pipeline using the already prepared Document AI model build, streams, and tasks. The pipeline will extract information from new inspection documents stored in an internal stage.

To create a processing pipeline:

- Set up the pipeline using streams and tasks.
- Upload new documents to an internal stage.
- View the extracted information.

Set up the processing pipeline

1. Create an internal my_pdf_stage stage to store the documents:

```
CREATE OR REPLACE STAGE my_pdf_stage
```

```
DIRECTORY = (ENABLE = TRUE)
```

```
ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE');
```

2. Create a my_pdf_stream stream on a my_pdf_stage stage:

```
CREATE STREAM my_pdf_stream ON STAGE my_pdf_stage;
```

3. Refresh the metadata of the directory table that will store the staged document files:

```
ALTER STAGE my_pdf_stage REFRESH;
```

4. Specify the database and schema:

```
USE DATABASE doc_ai_db;
```

```
USE SCHEMA doc_ai_schema;
```

5. Create a pdf_reviews table to store the information about the documents (such as file_name) and the data to be extracted from the PDF documents:

```
CREATE OR REPLACE TABLE pdf_reviews (
```

```
file_name VARCHAR,
```

```
file_size VARIANT,
```

```
last_modified VARCHAR,
```

```
snowflake_file_url VARCHAR,  
json_content VARCHAR  
);
```

The json_content column will include the extracted information in JSON format.

6. Create a load_new_file_data task to process new documents in the stage:

```
CREATE OR REPLACE TASK load_new_file_data  
  
  WAREHOUSE = <your_warehouse>  
  
  SCHEDULE = '1 minutes'  
  
  COMMENT = 'Process new files in the stage and insert data into the pdf_reviews  
table.'  
  
  WHEN SYSTEM$STREAM_HAS_DATA('my_pdf_stream')  
  
  AS  
  
  INSERT INTO pdf_reviews (  
  
    SELECT  
  
      RELATIVE_PATH AS file_name,  
  
      size AS file_size,  
  
      last_modified,  
  
      file_url AS snowflake_file_url,  
  
      inspection_reviews!PREDICT(GET_PRE_SIGNED_URL('@my_pdf_stage',  
RELATIVE_PATH), 1) AS json_content  
  
    FROM my_pdf_stream  
  
    WHERE METADATA$ACTION = 'INSERT'  
  
  );
```

Note that newly created tasks are automatically suspended.

7. Start the newly created task:

```
ALTER TASK load_new_file_data RESUME;
```


Upload new documents to an internal stage

1. To obtain the documents required for the tutorial, download the [zip file](#) to your local file system.
2. Unzip the content, which includes PDF files.
3. Sign in to [Snowsight](#).
4. In the navigation menu, select **Catalog » Database Explorer**.
5. Select the doc_ai_db database, the doc_ai_schema, and the my_pdf_stage stage.
6. Select **+ Files**.
7. In the **Upload Your Files** dialog that appears, select the files you just downloaded.
8. Select **Upload**.

View the extracted information

1. After uploading the documents to the stage, view the information extracted from new documents:

```
SELECT * FROM pdf_reviews;
```

2. Create a pdf_reviews_2 table to analyze the extracted information in separate columns:

```
CREATE OR REPLACE TABLE doc_ai_db.doc_ai_schema.pdf_reviews_2 AS (  
  WITH temp AS (  
    SELECT  
      RELATIVE_PATH AS file_name,  
      size AS file_size,  
      last_modified,  
      file_url AS snowflake_file_url,  
      inspection_reviews!PREDICT(get_presigned_url('@my_pdf_stage', RELATIVE_PATH),  
1) AS json_content  
    FROM directory(@my_pdf_stage)  
  )  
)
```

```

SELECT
    file_name,
    file_size,
    last_modified,
    snowflake_file_url,
    json_content:__documentMetadata.ocrScore::FLOAT AS ocrScore,
    f.value:score::FLOAT AS inspection_date_score,
    f.value:value::STRING AS inspection_date_value,
    g.value:score::FLOAT AS inspection_grade_score,
    g.value:value::STRING AS inspection_grade_value,
    i.value:score::FLOAT AS inspector_score,
    i.value:value::STRING AS inspector_value,
    ARRAY_TO_STRING(ARRAY_AGG(j.value:value::STRING), ', ') AS list_of_units
FROM temp,
    LATERAL FLATTEN(INPUT => json_content:inspection_date) f,
    LATERAL FLATTEN(INPUT => json_content:inspection_grade) g,
    LATERAL FLATTEN(INPUT => json_content:inspector) i,
    LATERAL FLATTEN(INPUT => json_content:list_of_units) j
GROUP BY ALL
);

```

3. View the output:

```
SELECT * FROM pdf_reviews_2;
```

The table uses the [FLATTEN](#) function to parse the json_content JSON into separate columns for easier viewing.

The table contains the data values (such as inspection_grade_value, inspection_date_value) that were defined when the model build was prepared for inspection documents, and the corresponding confidence scores (inspection_grade_score, inspection_date_score).

ANALYTICS WITH ANAND